

Rapport de Projet

Blockchain pour la Gestion Décentralisée
des Ressources dans une Smart City

Sous-Projet 4 : Smart Parking & EV Charging

**Reda Wafik
Ibrahim Berradi
Ayoub Najjout**

Faculté des Sciences et Techniques de Tanger
Master IASD



Module : Blockchain

Pr. Ikram BEN ABDEL OUAHAB

Année universitaire 2025-2026

Table des matières

Introduction Générale	5
1 Contexte et Problématique	6
1.1 Contexte Général des Smart Cities	6
1.2 Problématique du Stationnement Urbain	6
1.2.1 Les Défis Actuels	6
1.2.2 Limitations des Systèmes Traditionnels	6
1.3 La Révolution des Véhicules Électriques	6
1.3.1 Croissance du Marché EV	6
1.3.2 Défis de l'Infrastructure de Recharge	7
1.4 Apport de la Blockchain	7
1.4.1 Caractéristiques Fondamentales	7
1.4.2 Hyperledger Fabric : Blockchain d'Entreprise	7
1.5 Solution Proposée	7
2 Modélisation et Étude Technique	8
2.1 Architecture Globale du Système	8
2.1.1 Vue d'Ensemble	8
2.1.2 Réseau Hyperledger Fabric	8
2.2 Modélisation des Entités	8
2.2.1 Modèle de Données	8
2.3 Conception du Smart Contract	9
2.3.1 Structure du Chaincode	9
2.3.2 Logique de Réservation Critique	10
2.3.3 Mécanisme MVCC (Multi-Version Concurrency Control)	12
2.3.4 Cycle de Vie d'une Réservation	13
2.4 Sécurité et Prévention des Attaques	13
2.4.1 Validation des Smart Contracts	13
2.4.2 Module SOC : Security Operations Center	14
2.5 Backend : API et Gestion des Événements	15
2.5.1 Architecture Backend	15
2.5.2 Service Blockchain	16
2.5.3 Écoute des Événements Blockchain	17
3 Technologies Utilisées et Démonstration	19
3.1 Stack Technologique	19
3.1.1 Technologies Blockchain	19
3.1.2 Backend	19
3.1.3 Frontend	19
3.2 Démonstration de l'Application	19
3.2.1 Interface Principale : Dashboard	19
3.2.2 Création d'un Parking	20
3.2.3 Visualisation 3D du Parking	21
3.2.4 Enregistrement d'un Véhicule	21
3.2.5 Stationnement Automatique	22
3.2.6 Prévention de Double Réservation	22
3.2.7 Historique des Paiements	23

3.2.8	Métriques et Statistiques Blockchain	24
3.2.9	Processus de Réservation Complet	24
3.3	Module SOC : Détection d'Attaques	25
3.3.1	Dashboard du SOC	25
3.3.2	Simulation d'Attaque par Force Brute	26
3.3.3	Détection et Blocage	27
3.3.4	Registre des Incidents	27
3.3.5	Journal des Événements	28
3.4	Analyse de Performance	28
3.4.1	Méthodologie de Test	28
3.4.2	Métriques Mesurées	28
3.4.3	Résultats et Observations	29
3.4.4	Impact des Paramètres Blockchain	29
3.5	Traçabilité et Audit	29
3.5.1	Historique Complet des Transactions	29
3.5.2	Exemple de Traçabilité d'une Réservation	29
	Conclusion	31

Table des figures

1	Architecture en couches du système	8
2	Prévention de double réservation via MVCC	12
3	Dashboard principal de l'application Smart Parking	20
4	Formulaire de création d'un parking	20
5	Message de confirmation de création de parking	21
6	Visualisation 3D du parking avec détails des places	21
7	Formulaire d'enregistrement d'un véhicule EV	22
8	Véhicule stationné dans la visualisation 3D	22
9	Message d'échec lors de tentative de double réservation	23
10	Historique complet des paiements	23
11	Détails des 5 derniers paiements effectués	24
12	Statistiques et métriques de la blockchain	24
13	Création d'un véhicule non encore stationné	25
14	Réservation et paiement automatique réussis	25
15	Dashboard du Security Operations Center	26
16	Exécution du script d'attaque par force brute	26
17	Détection et affichage de l'adresse IP malveillante	27
18	Registre des incidents de sécurité	27
19	Journal des événements avec détails des tentatives d'attaque	28

Introduction Générale

Dans un contexte urbain en constante évolution, les villes intelligentes (Smart Cities) représentent une réponse innovante aux défis de mobilité, d'énergie et de gestion des ressources. La recherche d'une place de stationnement constitue l'un des problèmes majeurs des zones urbaines modernes, générant du trafic inutile, de la pollution et une perte de temps considérable pour les conducteurs.

L'émergence des véhicules électriques (EV) ajoute une nouvelle dimension à cette problématique, nécessitant non seulement des places de stationnement, mais également des infrastructures de recharge adaptées. La gestion centralisée traditionnelle de ces ressources présente plusieurs limitations : manque de transparence, risques de fraude (double réservation), absence de traçabilité et vulnérabilités de sécurité.

La technologie blockchain offre une solution prometteuse pour répondre à ces défis. En tant que registre distribué et immuable, elle garantit la transparence des transactions, prévient la fraude grâce à des mécanismes de consensus, et assure une traçabilité complète de toutes les opérations. Dans le cadre de ce projet, nous explorons l'application de la blockchain Hyperledger Fabric pour créer un système décentralisé de gestion de stationnement intelligent et de recharge pour véhicules électriques.

Objectifs du Projet

Ce projet s'inscrit dans le cadre plus large de la simulation d'une Smart City décentralisée et vise à :

- Concevoir et implémenter un système de stationnement intelligent basé sur blockchain
- Prévenir les doubles réservations grâce au mécanisme MVCC de Hyperledger Fabric
- Gérer les stations de recharge pour véhicules électriques de manière sécurisée
- Assurer la traçabilité complète des transactions et paiements
- Développer une interface utilisateur interactive avec visualisation 3D en temps réel
- Analyser les performances du système et expérimenter avec les paramètres blockchain
- Intégrer un module de sécurité (SOC) pour la détection et prévention des attaques

Structure du Rapport

Ce rapport est organisé en trois chapitres principaux :

Chapitre 1 présente le contexte général du projet, la problématique des systèmes de stationnement traditionnels, et l'apport de la blockchain dans ce domaine.

Chapitre 2 détaille la modélisation et l'étude technique du système, incluant l'architecture blockchain, la conception des smart contracts, et l'analyse des mécanismes de sécurité.

Chapitre 3 explore les technologies utilisées, présente une démonstration complète de l'application développée, et analyse les résultats obtenus en termes de performance et de sécurité.

1 Contexte et Problématique

1.1 Contexte Général des Smart Cities

Les villes intelligentes émergent comme une réponse aux défis urbains du XXI^e siècle. Avec une urbanisation croissante, plus de 60% de la population mondiale devrait vivre en zones urbaines d'ici 2030. Cette concentration démographique engendre des problèmes complexes de gestion des ressources, de mobilité et d'infrastructures.

Une Smart City utilise les technologies de l'information et de la communication (TIC) pour optimiser la gestion des services urbains : transport, énergie, sécurité, et environnement. Cependant, la centralisation des données et des systèmes de contrôle pose des risques significatifs en termes de sécurité, de confidentialité et de fiabilité.

1.2 Problématique du Stationnement Urbain

1.2.1 Les Défis Actuels

Le stationnement en milieu urbain représente un défi majeur pour plusieurs raisons :

1. **Perte de temps** : Les conducteurs passent en moyenne 20 minutes à chercher une place de stationnement
2. **Impact environnemental** : 30% du trafic urbain est généré par la recherche de stationnement
3. **Congestion** : La circulation inutile augmente les embouteillages
4. **Émissions de CO₂** : La recherche de stationnement génère des émissions polluantes considérables

1.2.2 Limitations des Systèmes Traditionnels

Les systèmes de stationnement centralisés présentent plusieurs vulnérabilités :

- **Double réservation** : Possibilité de réserver la même place plusieurs fois simultanément
- **Manque de transparence** : Absence de traçabilité des transactions
- **Point de défaillance unique** : Vulnérabilité des serveurs centralisés
- **Fraude aux paiements** : Difficultés de vérification et d'audit
- **Latence** : Délais de synchronisation entre différents points d'accès

1.3 La Révolution des Véhicules Électriques

1.3.1 Croissance du Marché EV

Le marché des véhicules électriques connaît une croissance exponentielle, avec des projections estimant à 145 millions le nombre de véhicules électriques en circulation d'ici 2030. Cette transition énergétique nécessite une infrastructure de recharge adaptée et intelligente.

1.3.2 Défis de l'Infrastructure de Recharge

L'infrastructure de recharge pour véhicules électriques présente des défis spécifiques :

- Gestion de la disponibilité des bornes de recharge
- Optimisation du temps de recharge
- Coordination avec le réseau électrique
- Tarification dynamique basée sur la demande
- Compatibilité entre différents types de véhicules

1.4 Apport de la Blockchain

1.4.1 Caractéristiques Fondamentales

La technologie blockchain offre des propriétés essentielles pour résoudre les problèmes identifiés :

1. **Décentralisation** : Élimination du point de défaillance unique
2. **Immuabilité** : Les transactions enregistrées ne peuvent être modifiées
3. **Transparence** : Toutes les transactions sont auditable
4. **Sécurité cryptographique** : Protection contre la falsification
5. **Consensus distribué** : Validation collective des transactions

1.4.2 Hyperledger Fabric : Blockchain d'Entreprise

Hyperledger Fabric est une plateforme blockchain permissionnée particulièrement adaptée aux cas d'usage d'entreprise :

- **Réseau privé** : Contrôle d'accès et identités gérées
- **Smart Contracts (Chaincode)** : Logique métier programmable
- **Canaux privés** : Communication confidentielle entre organisations
- **Mécanisme MVCC** : Prévention automatique des conflits de concurrence
- **Performance** : Capable de traiter des milliers de transactions par seconde

1.5 Solution Proposée

Notre projet propose un système complet de gestion de stationnement intelligent et de recharge EV basé sur Hyperledger Fabric, intégrant :

1. Un **smart contract** gérant les réservations, paiements et cycle de vie du stationnement
2. Un **backend Node.js** avec API REST et communication temps réel (WebSocket)
3. Une **interface web React** avec visualisation 3D interactive (Three.js)
4. Un **module SOC** (Security Operations Center) pour la détection d'attaques
5. Des **mécanismes de prévention** contre les doubles réservations et fraudes

Cette solution garantit la transparence, la sécurité et l'efficacité de la gestion des ressources de stationnement dans un contexte de Smart City.

2 Modélisation et Étude Technique

2.1 Architecture Globale du Système

2.1.1 Vue d'Ensemble

L'architecture du système suit un modèle en couches, séparant clairement les responsabilités entre la blockchain, le backend applicatif et l'interface utilisateur.

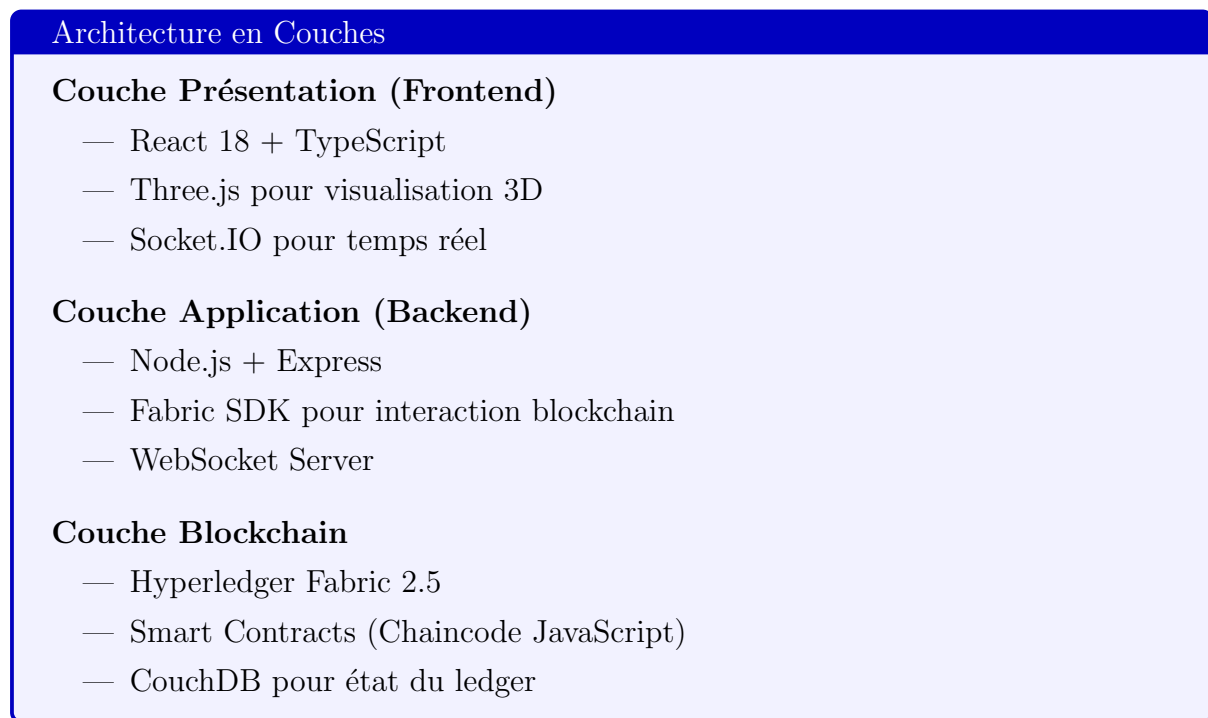


FIGURE 1 – Architecture en couches du système

2.1.2 Réseau Hyperledger Fabric

Le réseau blockchain est composé des éléments suivants :

- **Orderer** : Service d'ordonnancement utilisant le consensus Raft
- **Peer Org1** : Nœud appartenant à l'organisation 1
- **Peer Org2** : Nœud appartenant à l'organisation 2
- **CouchDB** : Base de données d'état pour requêtes riches
- **CA (Certificate Authority)** : Gestion des identités et certificats

2.2 Modélisation des Entités

2.2.1 Modèle de Données

Le système gère quatre types d'entités principales stockées sur la blockchain :

1. **Parking** : Représente un parking physique

```

1 {
2   parkingId: "P1",
3   name: "Campus Parking",

```



```
4   location: "Zone A",
5   totalPlaces: 20,
6   evStations: 5,
7   docType: "parking"
8 }
```

Listing 1 – Structure de l'entité Parking

2. **Place** : Une place de stationnement individuelle

```
1 {
2   placeId: "PL1",
3   parkingId: "P1",
4   type: "ev", // "regular" ou "ev"
5   status: "free", // "free", "reserved", "occupied"
6   currentCarId: null,
7   lastUpdated: "2025-12-28T10:30:00.000Z",
8   docType: "place"
9 }
```

Listing 2 – Structure de l'entité Place

3. **Car** : Un véhicule enregistré dans le système

```
1 {
2   carId: "CAR1",
3   owner: "Alice",
4   batteryLevel: 80,
5   evCompatible: true,
6   parkingId: "P1",
7   docType: "car"
8 }
```

Listing 3 – Structure de l'entité Car

4. **Reservation** : Une réservation de place

```
1 {
2   reservationId: "RES-1735387200-123456",
3   carId: "CAR1",
4   placeId: "PL1",
5   parkingId: "P1",
6   startTime: "2025-12-28T10:00:00.000Z",
7   endTime: null,
8   paid: false,
9   active: true,
10  amount: 10.50,
11  parkingStarted: null,
12  txId: "abc123...",
13  docType: "reservation"
14 }
```

Listing 4 – Structure de l'entité Reservation

2.3 Conception du Smart Contract

2.3.1 Structure du Chaincode

Le chaincode est développé en JavaScript (Node.js) et hérite de la classe **Contract** fournie par Fabric. Il expose plusieurs fonctions transactionnelles et de requête.

```

1 const { Contract } = require('fabric-contract-api');
2
3 class ParkingContract extends Contract {
4   async InitLedger(ctx) {
5     console.info('===== START : Initialize Ledger
6       =====');
7
8     const initTimestamp = '2025-12-28T00:00:00.000Z';
9
10    // Creation du parking par défaut
11    const parking = {
12      parkingId: 'P1',
13      name: 'Campus Parking',
14      location: 'Zone A',
15      totalPlaces: 20,
16      evStations: 5,
17      docType: 'parking'
18    };
19
20    await ctx.stub.putState(parking.parkingId,
21      Buffer.from(JSON.stringify(parking)));
22
23    // Creation des places
24    for (let i = 1; i <= 20; i++) {
25      const place = {
26        placeId: 'PL${i}',
27        parkingId: 'P1',
28        type: i <= 5 ? 'ev' : 'regular',
29        status: 'free',
30        currentCarId: null,
31        lastUpdated: initTimestamp,
32        docType: 'place'
33      };
34      await ctx.stub.putState(place.placeId,
35        Buffer.from(JSON.stringify(place)));
36    }
37
38    return JSON.stringify({
39      success: true,
40      message: 'Ledger initialized'
41    });
42  }

```

Listing 5 – Initialisation du Contrat

2.3.2 Logique de Réservation Critique

La fonction de réservation constitue le cœur du système et implémente la prévention des doubles réservations grâce au mécanisme MVCC de Fabric.

```

1 async RequestReservation(ctx, carId, parkingId, desiredType) {
2   console.info('===== START : Request Reservation
3     =====');
4
5   // Verification de l'existence de la voiture
6   const carExists = await this._assetExists(ctx, carId);
7   if (!carExists) {

```

```

7      throw new Error('Car ${carId} does not exist');
8  }
9
10 // Recherche d'une place disponible
11 const queryString = {
12     selector: {
13         docType: 'place',
14         parkingId: parkingId,
15         type: desiredType,
16         status: 'free'
17     }
18 };
19
20 const iterator = await ctx.stub.getQueryResult(
21     JSON.stringify(queryString)
22 );
23 const result = await iterator.next();
24
25 if (result.done) {
26     throw new Error(
27         'No available ${desiredType} places in parking ${parkingId}
28         ');
29 }
30
31 const place = JSON.parse(result.value.value.toString());
32 await iterator.close();
33
34 // OPERATION ATOMIQUE : Reservation de la place
35 // Le mecanisme MVCC de Fabric previent automatiquement
36 // les doubles reservations
37 place.status = 'reserved';
38 place.currentCarId = carId;
39 place.lastUpdated = this._getTxTimestampString(ctx);
40
41 await ctx.stub.putState(place.placeId,
42     Buffer.from(JSON.stringify(place)));
43
44 // Creation de l'enregistrement de reservation
45 const txTimestamp = ctx.stub.getTxTimestamp();
46 const reservationId = 'RES-${txTimestamp.seconds.toNumber()}-${
47     txTimestamp.nanos}';
48 const reservation = {
49     reservationId,
50     carId,
51     placeId: place.placeId,
52     parkingId,
53     startTime: this._getTxTimestampString(ctx),
54     endTime: null,
55     paid: false,
56     active: true,
57     txId: ctx.stub.getTxID(),
58     docType: 'reservation'
59 };
60
61 await ctx.stub.putState(reservationId,
62     Buffer.from(JSON.stringify(reservation)));
63
64 // Emission d'evenement

```

```

64     const event = {
65         type: 'ReservationCreated',
66         reservationId,
67         carId,
68         placeId: place.placeId,
69         parkingId,
70         txId: ctx.stub.getTxID()
71     };
72     await ctx.stub.setEvent('ReservationCreated',
73         Buffer.from(JSON.stringify(event)));
74
75     return JSON.stringify({
76         reservation,
77         place,
78         txId: ctx.stub.getTxID()
79     });
80 }

```

Listing 6 – Fonction de Réservation (partie critique)

2.3.3 Mécanisme MVCC (Multi-Version Concurrency Control)

Le mécanisme MVCC est crucial pour prévenir les doubles réservations. Voici son fonctionnement :

1. Chaque état dans le ledger possède une **version**
2. Lorsqu'une transaction lit un état, elle note sa version actuelle
3. Lors du commit, Fabric vérifie si la version a changé
4. Si la version a changé (modification concurrente), la transaction échoue avec **MVCC_READ_CONFLICT**
5. Seule la première transaction peut commiter avec succès

Scénario de Double Réservation

Temps T0 : Place PL1 - Version 1 - Status : "free"

Temps T1 :

- Transaction A lit PL1 (Version 1)
- Transaction B lit PL1 (Version 1)

Temps T2 :

- Transaction A tente de réserver PL1
- Fabric met à jour : PL1 - Version 2 - Status : "reserved"
- Transaction A : **SUCCESS**

Temps T3 :

- Transaction B tente de réserver PL1
- Fabric détecte : Version lue (1) $\neq$ Version actuelle (2)
- Transaction B : **MVCC_READ_CONFLICT**

FIGURE 2 – Prévention de double réservation via MVCC

2.3.4 Cycle de Vie d'une Réservation

Une réservation passe par plusieurs états définis :

1. **Création** : Réservation initiale, place marquée "reserved"
2. **Paiement** : Confirmation du paiement, `paid = true`
3. **Début stationnement** : Véhicule garé, place marquée "occupied"
4. **Fin stationnement** : Véhicule parti, place marquée "free", `active = false`

```
1 async ConfirmPayment(ctx, reservationId, amount) {
2   const resAsBytes = await ctx.stub.getState(reservationId);
3   if (!resAsBytes || resAsBytes.length === 0) {
4     throw new Error('Reservation ${reservationId} does not exist');
5   }
6
7   const reservation = JSON.parse(resAsBytes.toString());
8
9   if (reservation.paid) {
10    throw new Error('Reservation ${reservationId} is already paid');
11  }
12
13  if (!reservation.active) {
14    throw new Error('Reservation ${reservationId} is not active');
15  }
16
17  // Mise a jour de la reservation
18  reservation.paid = true;
19  reservation.amount = parseFloat(amount);
20  reservation.paymentTime = this._getTxTimestampString(ctx);
21  reservation.paymentTxId = ctx.stub.getTxID();
22
23  await ctx.stub.putState(reservationId,
24    Buffer.from(JSON.stringify(reservation)));
25
26  // Emission d'événement
27  const event = {
28    type: 'PaymentConfirmed',
29    reservationId,
30    amount: parseFloat(amount),
31    txId: ctx.stub.getTxID()
32  };
33  await ctx.stub.setEvent('PaymentConfirmed',
34    Buffer.from(JSON.stringify(event)));
35
36  return JSON.stringify(reservation);
37 }
```

Listing 7 – Fonction de Confirmation de Paiement

2.4 Sécurité et Prévention des Attaques

2.4.1 Validation des Smart Contracts

Le chaincode implémente plusieurs niveaux de validation pour garantir l'intégrité des transactions :

1. **Validation des prérequis** : Vérification de l'existence des entités référencées
2. **Validation des états** : Vérification de la cohérence des statuts
3. **Validation métier** : Application des règles métier spécifiques

```

1 async StartParking(ctx, reservationId) {
2   const resAsBytes = await ctx.stub.getState(reservationId);
3   if (!resAsBytes || resAsBytes.length === 0) {
4     throw new Error('Reservation ${reservationId} does not exist');
5   }
6
7   const reservation = JSON.parse(resAsBytes.toString());
8
9   // VALIDATION CRITIQUE : Le paiement doit etre confirme
10  if (!reservation.paid) {
11    throw new Error(
12      'Payment required before starting parking for reservation $
13      {reservationId}'
14    );
15  }
16
17  if (!reservation.active) {
18    throw new Error('Reservation ${reservationId} is not active');
19  }
20
21  // Mise a jour du statut de la place
22  const placeAsBytes = await ctx.stub.getState(reservation.placeId);
23  const place = JSON.parse(placeAsBytes.toString());
24
25  place.status = 'occupied';
26  place.lastUpdated = this._getTxTimestampString(ctx);
27
28  await ctx.stub.putState(reservation.placeId,
29    Buffer.from(JSON.stringify(place)));
30
31  // Mise a jour de la reservation
32  reservation.parkingStarted = this._getTxTimestampString(ctx);
33  reservation.parkingStartTxId = ctx.stub.getTxID();
34
35  await ctx.stub.putState(reservationId,
36    Buffer.from(JSON.stringify(reservation)));
37
38  return JSON.stringify({ reservation, place });
39 }

```

Listing 8 – Validation avant Début de Stationnement

2.4.2 Module SOC : Security Operations Center

En complément du système principal de parking, nous avons développé un module SOC dédié à la détection et à la prévention des attaques. Ce module surveille en temps réel les tentatives d'intrusion et les comportements suspects.

Types d'Attaques Détectées

1. Attaque par Force Brute

- Tentatives répétées de réservation sur des places déjà occupées

- Détection basée sur le nombre de tentatives échouées
- Blocage automatique de l'adresse IP après seuil dépassé

2. Tentative de Double Réserveation

- Réservations concurrentes sur la même place
- Détection via erreurs MVCC_READ_CONFLICT
- Enregistrement de l'incident dans le ledger

3. Fraude aux Paiements

- Tentative de stationnement sans paiement
- Détection par validation du chaincode
- Rejet automatique de la transaction

4. Modification Non Autorisée

- Tentative de modification directe de la base CouchDB
- Protection via hachage cryptographique
- Invalidation détectée lors de la prochaine lecture

Architecture du Module SOC

```
1 async DetectAttack(ctx, attackType, details) {
2   console.info('===== ATTACK DETECTED =====');
3
4   const txTimestamp = ctx.stub.getTxTimestamp();
5   const attackRecord = {
6     attackId: 'ATK-${txTimestamp.seconds.toNumber()}-${txTimestamp.
7       nanos}',
8     attackType,
9     details,
10    timestamp: this._getTxTimestampString(ctx),
11    txId: ctx.stub.getTxID(),
12    docType: 'attack'
13  };
14
15  await ctx.stub.putState(attackRecord.attackId,
16    Buffer.from(JSON.stringify(attackRecord)));
17
18  // Emission d'événement pour alerter le SOC
19  const event = {
20    type: 'AttackDetected',
21    attackType,
22    details,
23    txId: ctx.stub.getTxID()
24  };
25  await ctx.stub.setEvent('AttackDetected',
26    Buffer.from(JSON.stringify(event)));
27
28  return JSON.stringify(attackRecord);
29 }
```

Listing 9 – Détection d'Attaque dans le Chaincode

2.5 Backend : API et Gestion des Événements

2.5.1 Architecture Backend

Le backend est développé en Node.js avec TypeScript et suit une architecture modulaire :

- **Services** : Logique métier et interaction avec Fabric SDK
- **Controllers** : Gestion des requêtes HTTP
- **Routes** : Définition des endpoints API REST
- **WebSocket Server** : Communication temps réel avec le frontend

2.5.2 Service Blockchain

```
1 import { Gateway, Network, Contract } from 'fabric-network';
2 import FabricCAServices from 'fabric-ca-client';
3 import path from 'path';
4 import fs from 'fs';
5
6 class BlockchainService {
7   private gateway: Gateway;
8   private network: Network;
9   private contract: Contract;
10
11   async connect() {
12     // Configuration du wallet
13     const walletPath = path.join(__dirname, '../..../wallet');
14     const wallet = await Wallets.newFileSystemWallet(walletPath);
15
16     // Chargement du profil de connexion
17     const ccpPath = path.resolve(
18       __dirname,
19       '../..../fabric-samples/test-network/organizations/
20         peerOrganizations/org1.example.com/connection-org1.json'
21     );
22     const ccp = JSON.parse(fs.readFileSync(ccpPath, 'utf8'));
23
24     // Creation de l'identite admin
25     const identity = await wallet.get('admin');
26     if (!identity) {
27       await this.enrollAdmin(wallet, ccp);
28     }
29
30     // Connexion a la gateway
31     this.gateway = new Gateway();
32     await this.gateway.connect(ccp, {
33       wallet,
34       identity: 'admin',
35       discovery: { enabled: true, asLocalhost: true }
36     });
37
38     // Acces au reseau et au contrat
39     this.network = await this.gateway.getNetwork('mychannel');
40     this.contract = this.network.getContract('parking');
41
42     console.log('Connected to Fabric Gateway');
43   }
44
45   async createCar(carId: string, owner: string, batteryLevel: number,
46     evCompatible: boolean, parkingId: string) {
47     const result = await this.contract.submitTransaction(
48       'RegisterCar',
49       carId,
```



```

50         batteryLevel.toString(),
51         evCompatible.toString(),
52         parkingId
53     );
54     return JSON.parse(result.toString());
55 }
56
57 async requestReservation(carId: string, parkingId: string,
58                         desiredType: string) {
59     const result = await this.contract.submitTransaction(
60         'RequestReservation',
61         carId,
62         parkingId,
63         desiredType
64     );
65     return JSON.parse(result.toString());
66 }
67
68 async confirmPayment(reservationId: string, amount: number) {
69     const result = await this.contract.submitTransaction(
70         'ConfirmPayment',
71         reservationId,
72         amount.toString()
73     );
74     return JSON.parse(result.toString());
75 }
76 }
77
78 export default new BlockchainService();

```

Listing 10 – Service d'Interaction avec Fabric

2.5.3 Écoute des Événements Blockchain

Le backend écoute en temps réel les événements émis par le chaincode et les diffuse aux clients via WebSocket.

```

1 import { io } from '../index';
2
3 class EventService {
4     async startEventListeners(network: Network) {
5         const contract = network.getContract('parking');
6
7         // Ecoute des evenements de reservation
8         await contract.addContractListener(
9             'ReservationCreated',
10            'reservation-listener',
11            (event) => {
12                const payload = JSON.parse(event.payload.toString());
13                console.log('Reservation created:', payload);
14
15                // Diffusion vers tous les clients WebSocket
16                io.emit('reservationCreated', payload);
17            }
18        );
19
20        // Ecoute des evenements de paiement
21        await contract.addContractListener(
22            'PaymentConfirmed',

```

```
23     'payment-listener',
24     (event) => {
25         const payload = JSON.parse(event.payload.toString());
26         console.log('Payment confirmed:', payload);
27         io.emit('paymentConfirmed', payload);
28     }
29 );
30
31 // Ecoute des evenements de parking
32 await contract.addContractListener(
33     'ParkingStarted',
34     'parking-start-listener',
35     (event) => {
36         const payload = JSON.parse(event.payload.toString());
37         console.log('Parking started:', payload);
38         io.emit('parkingStarted', payload);
39     }
40 );
41
42 // Ecoute des attaques
43 await contract.addContractListener(
44     'AttackDetected',
45     'attack-listener',
46     (event) => {
47         const payload = JSON.parse(event.payload.toString());
48         console.log('Attack detected:', payload);
49         io.emit('attackDetected', payload);
50     }
51 );
52
53 console.log('Event listeners started');
54 }
55 }
56
57 export default new EventService();
```

Listing 11 – Service de Gestion des Événements

3 Technologies Utilisées et Démonstration

3.1 Stack Technologique

3.1.1 Technologies Blockchain

- **Hyperledger Fabric 2.5** : Plateforme blockchain permissionnée
- **JavaScript Chaincode** : Smart contracts en Node.js
- **CouchDB** : Base de données NoSQL pour l'état du ledger
- **Docker & Docker Compose** : Conteneurisation et orchestration
- **Fabric CA** : Autorité de certification pour gestion des identités

3.1.2 Backend

- **Node.js 18+** : Environnement d'exécution JavaScript
- **TypeScript** : Typage statique pour JavaScript
- **Express** : Framework web minimaliste
- **Fabric Node SDK** : Bibliothèque d'interaction avec Fabric
- **Socket.IO** : Communication bidirectionnelle en temps réel
- **Winston** : Bibliothèque de journalisation

3.1.3 Frontend

- **React 18** : Bibliothèque de construction d'interfaces utilisateur
- **TypeScript** : Sécurité des types
- **Three.js** : Bibliothèque de graphisme 3D WebGL
- **@react-three/fiber** : Renderer React pour Three.js
- **Socket.IO Client** : Réception des événements temps réel
- **Vite** : Outil de build ultra-rapide

3.2 Démonstration de l'Application

3.2.1 Interface Principale : Dashboard

L'interface principale (Figure 3) présente un tableau de bord complet permettant de :

- Créer un parking avec configuration des places normales et EV
- Enregistrer des véhicules avec spécification du niveau de batterie pour les EV
- Réserver une place pour un véhicule non encore stationné
- Visualiser les statistiques de paiements et transactions
- Accéder aux détails via navigation interactive

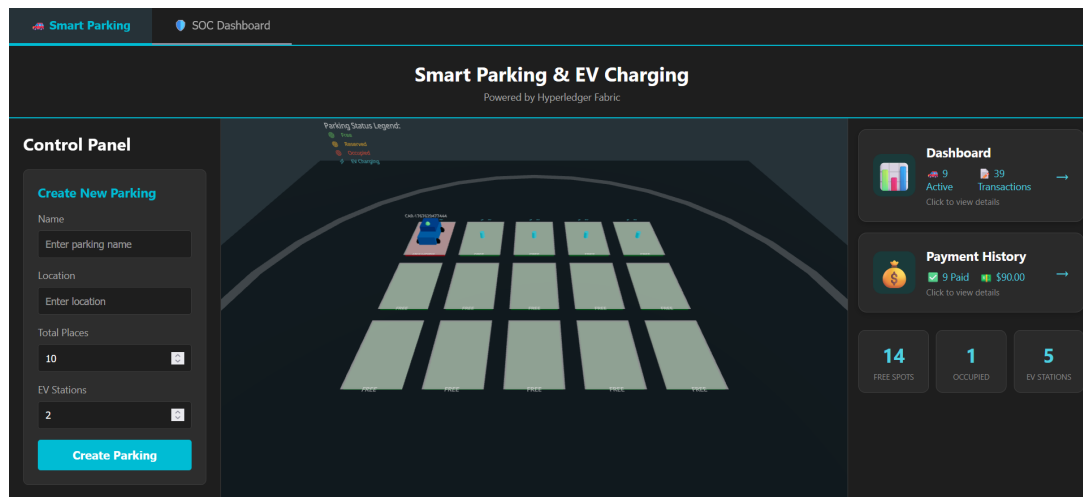


FIGURE 3 – Dashboard principal de l'application Smart Parking

3.2.2 Création d'un Parking

La Figure 4 montre le formulaire de création d'un parking. L'utilisateur peut spécifier :

- Nom et localisation du parking
- Nombre total de places
- Nombre de places équipées pour recharge EV

Dans cet exemple, un parking de 5 places est créé avec 2 places EV et 3 places normales.

FIGURE 4 – Formulaire de création d'un parking

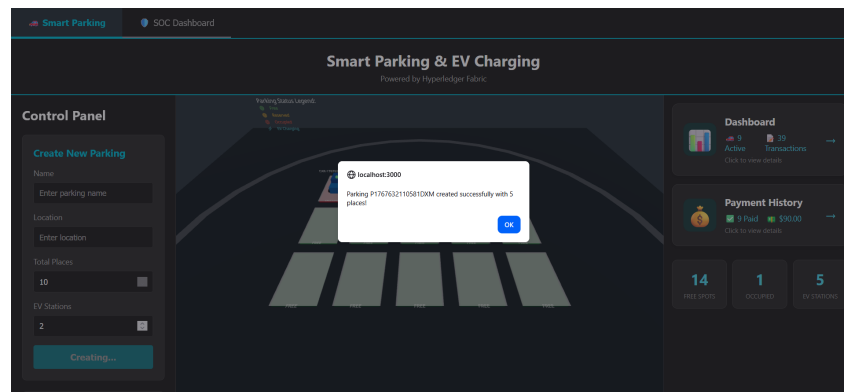


FIGURE 5 – Message de confirmation de création de parking

3.2.3 Visualisation 3D du Parking

Une fois le parking créé, l'utilisateur peut le sélectionner pour visualiser sa représentation 3D (Figure 6). Cette vue interactive affiche :

- Les places en 3D avec code couleur (vert = libre, orange = réservé, rouge = occupé)
- Les stations de recharge EV représentées par des sphères bleues
- Statistiques en temps réel : places disponibles, réservées et occupées
- Contrôles de rotation, zoom et panoramique

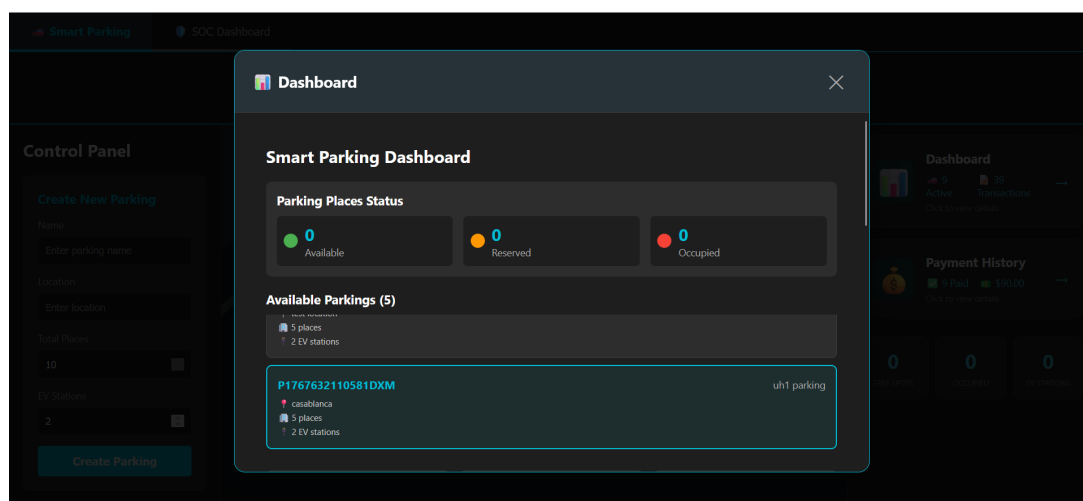


FIGURE 6 – Visualisation 3D du parking avec détails des places

3.2.4 Enregistrement d'un Véhicule

La Figure 7 illustre l'enregistrement d'un véhicule électrique dans le système. Les informations requises incluent :

- Identifiant du véhicule
- Propriétaire (dans cet exemple : "owner1")
- Compatibilité EV (case cochée)
- Niveau de batterie (70% dans cet exemple)
- Parking associé

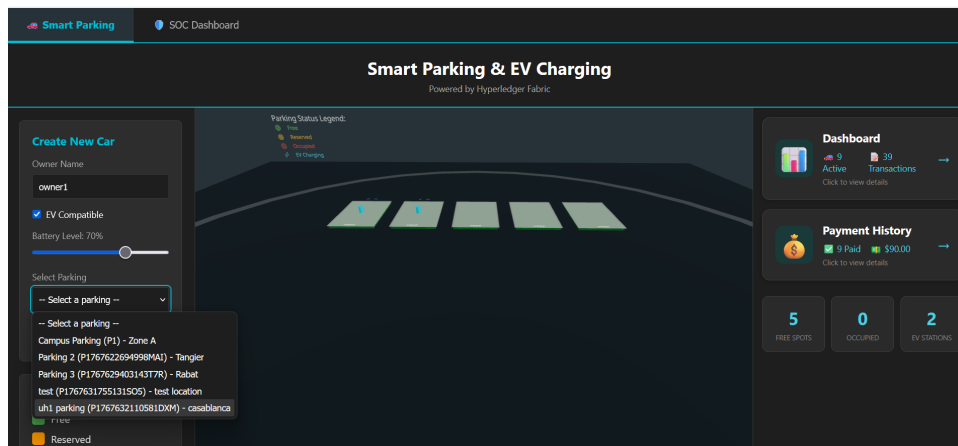


FIGURE 7 – Formulaire d'enregistrement d'un véhicule EV

3.2.5 Stationnement Automatique

Après enregistrement, le véhicule est automatiquement stationné dans une place EV disponible. La Figure 8 montre la mise à jour en temps réel de la visualisation 3D, où le véhicule apparaît dans son emplacement assigné.

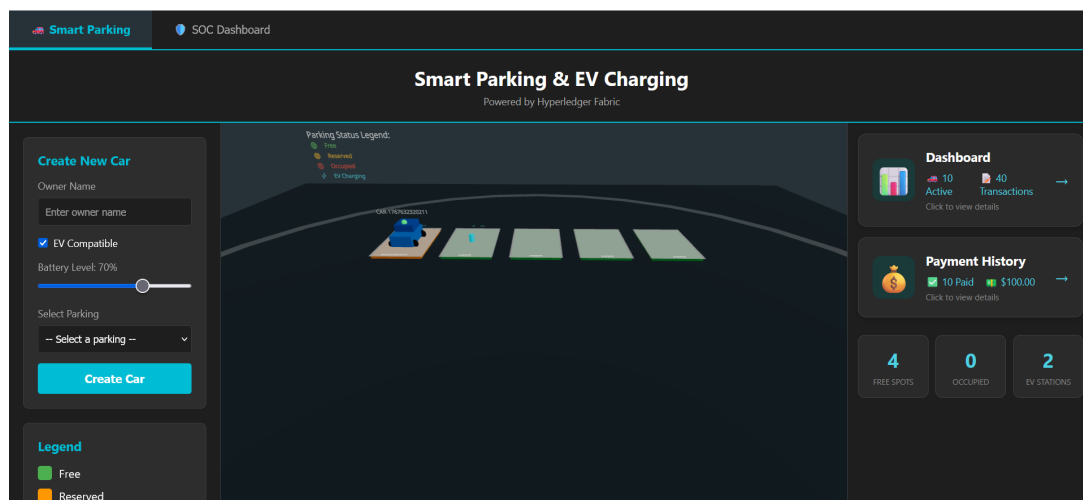


FIGURE 8 – Véhicule stationné dans la visualisation 3D

3.2.6 Prévention de Double Réservation

La Figure 9 démontre le mécanisme de prévention des doubles réservations. Lorsqu'un utilisateur tente de réserver une place EV déjà occupée, le système détecte le conflit via le mécanisme MVCC et affiche un message d'échec explicite.

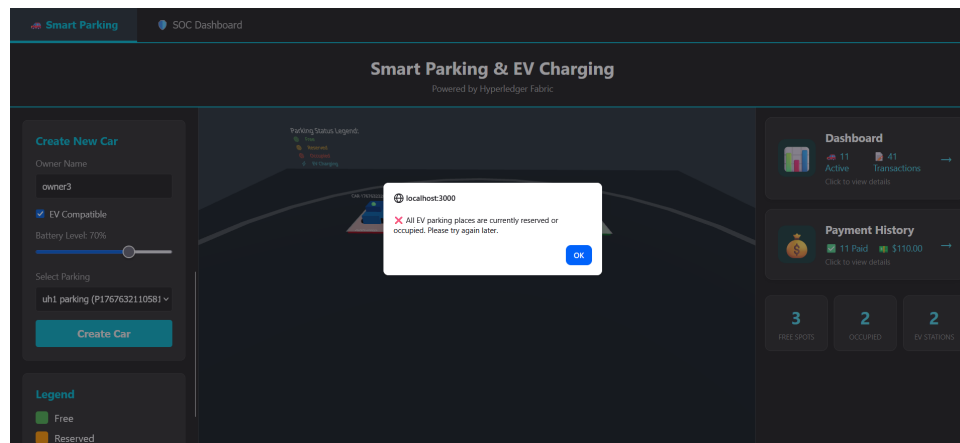


FIGURE 9 – Message d'échec lors de tentative de double réservation

Ce mécanisme garantit qu'une seule transaction peut réussir lorsque plusieurs utilisateurs tentent de réserver la même place simultanément.

3.2.7 Historique des Paiements

L'application maintient un historique complet de tous les paiements effectués (Figure 10). Cet historique est immuable grâce à la blockchain et permet un audit complet.

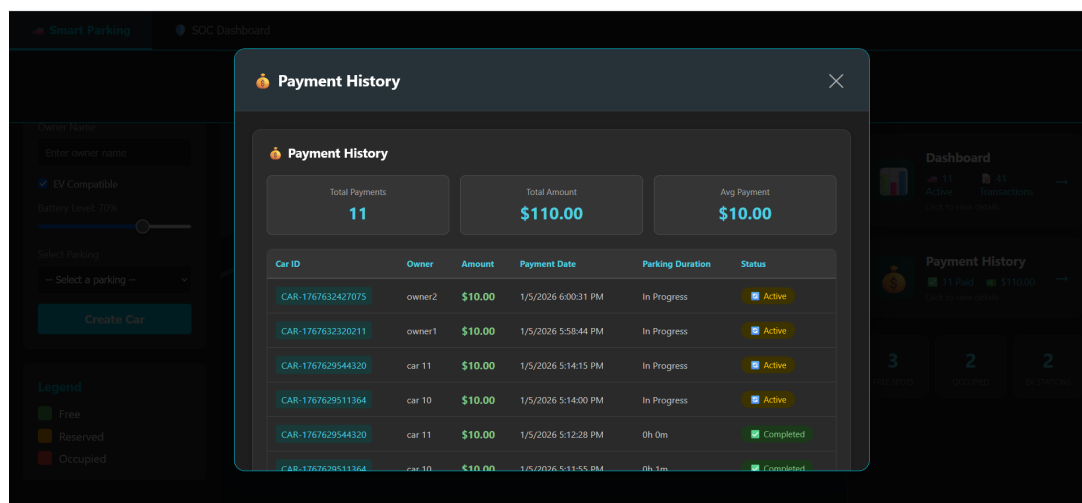


FIGURE 10 – Historique complet des paiements

La Figure 11 montre un zoom sur les 5 derniers paiements avec leurs informations détaillées : montant, date/heure, statut de confirmation, et identifiant de transaction blockchain.

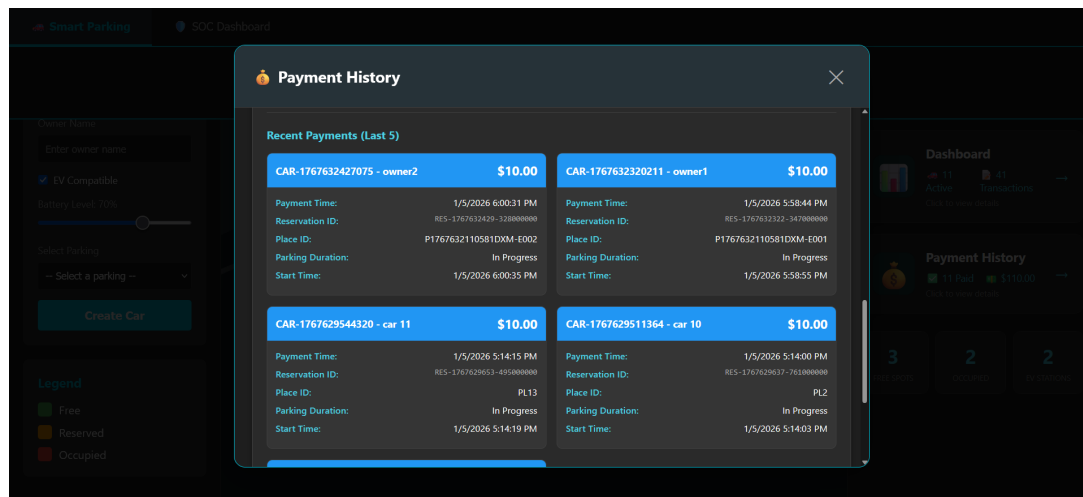


FIGURE 11 – Détails des 5 derniers paiements effectués

3.2.8 Métriques et Statistiques Blockchain

La Figure 12 présente les métriques de performance du système blockchain :

- **Nombre total de transactions** : Compteur de toutes les opérations
- **Latence moyenne** : Temps moyen de traitement des transactions
- **Taux de succès** : Pourcentage de transactions réussies
- **Liste des réservations** : Vue détaillée accessible par clic

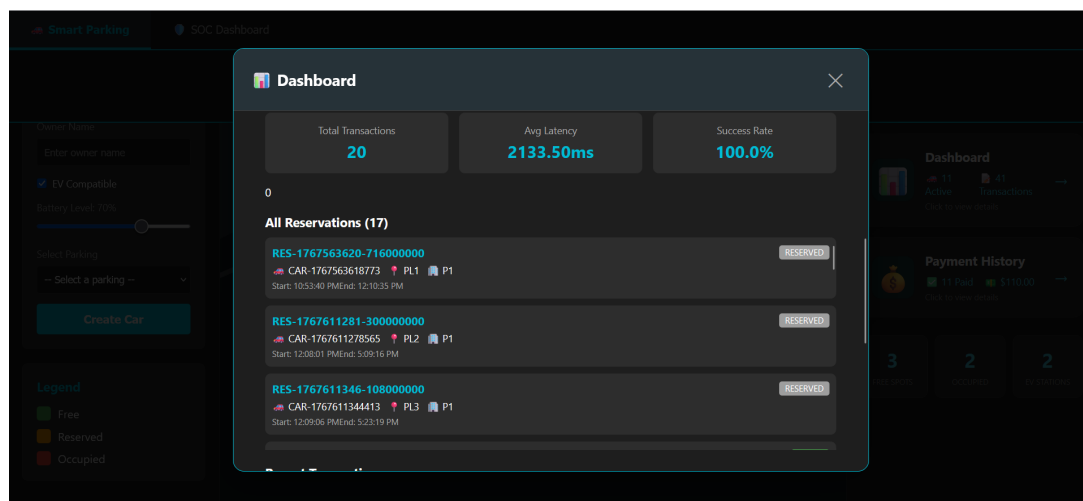


FIGURE 12 – Statistiques et métriques de la blockchain

3.2.9 Processus de Réservation Complet

Les Figures 13 et 14 illustrent le processus complet de réservation pour un véhicule non encore stationné :



FIGURE 13 – Création d'un véhicule non encore stationné

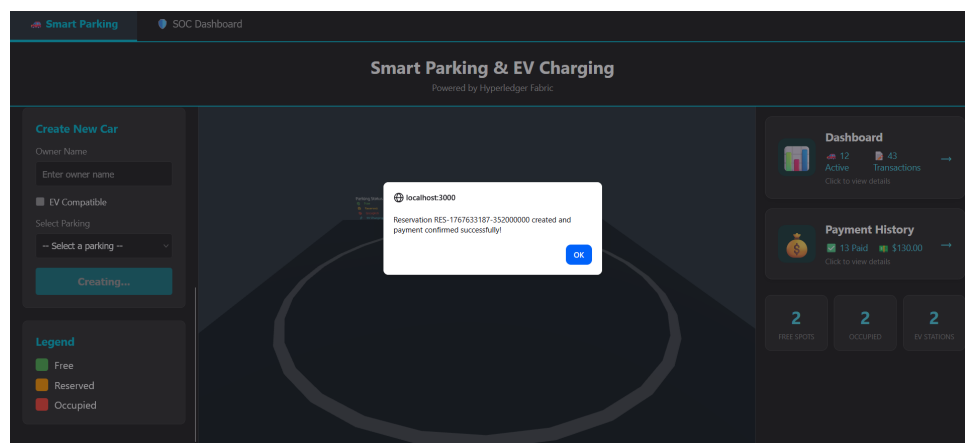


FIGURE 14 – Réservation et paiement automatique réussis

Le système effectue automatiquement :

1. Réservation de la place disponible
2. Traitement et confirmation du paiement (après 2 secondes)
3. Notification de succès en temps réel via WebSocket

3.3 Module SOC : Détection d'Attaques

3.3.1 Dashboard du SOC

Le module Security Operations Center fournit une vue complète de la sécurité du système (Figure 15) :

- Nombre total d'événements détectés
- Alertes de sécurité actives
- Incidents enregistrés
- Adresses IP bloquées

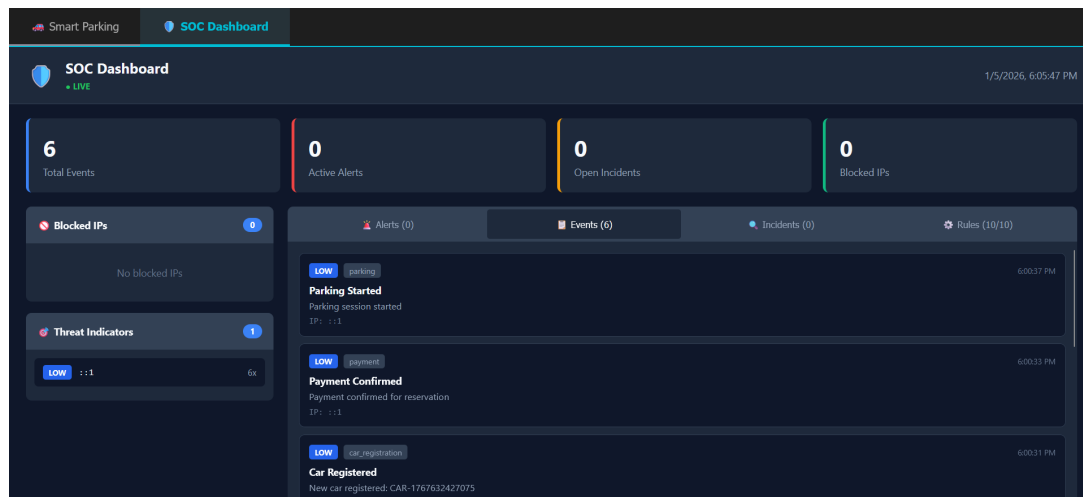


FIGURE 15 – Dashboard du Security Operations Center

3.3.2 Simulation d'Attaque par Force Brute

La Figure 16 montre l'exécution d'un script simulant une attaque par force brute. Le terminal affiche :

- Tentatives répétées de réservation
- Échecs successifs des transactions
- Message de fin d'attaque

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE
ibrahim_blockchain@DESKTOP-Q1LOFRT:~/smart-Parking-Ev$ cd scripts/soc-attacks/
ibrahim_blockchain@DESKTOP-Q1LOFRT:~/smart-Parking-Ev/scripts/soc-attacks$ ls
brute-force.js  node_modules  package.json  run-all-attacks.js
ddos-simulation.js  package-lock.json  payment-fraud.js
ibrahim_blockchain@DESKTOP-Q1LOFRT:~/smart-Parking-Ev/scripts/soc-attacks$ node brute-force.js
● Simulating Brute Force Attack...

Attempt 1/10: Trying to reserve place...
❌ Failed (expected)
Attempt 2/10: Trying to reserve place...
❌ Failed (expected)
Attempt 3/10: Trying to reserve place...
❌ Failed (expected)
Attempt 4/10: Trying to reserve place...
❌ Failed (expected)
Attempt 5/10: Trying to reserve place...
❌ Failed (expected)
Attempt 6/10: Trying to reserve place...
❌ Failed (expected)
Attempt 7/10: Trying to reserve place...
❌ Failed (expected)
Attempt 8/10: Trying to reserve place...
❌ Failed (expected)
Attempt 9/10: Trying to reserve place...
❌ Failed (expected)
Attempt 10/10: Trying to reserve place...
❌ Failed (expected)

✅ Brute force attack simulation complete!
Check SOC Dashboard for alerts 🚨

ibrahim_blockchain@DESKTOP-Q1LOFRT:~/smart-Parking-Ev/scripts/soc-attacks$

```

FIGURE 16 – Exécution du script d'attaque par force brute

3.3.3 Détection et Blocage

Suite à l'attaque, le dashboard SOC (Figure 17) affiche :

- La nouvelle adresse IP détectée comme malveillante
- Alerte indiquant le type d'attaque ("Brute Force")
- Bouton "Acknowledge" permettant à l'administrateur de marquer l'alerte comme vue

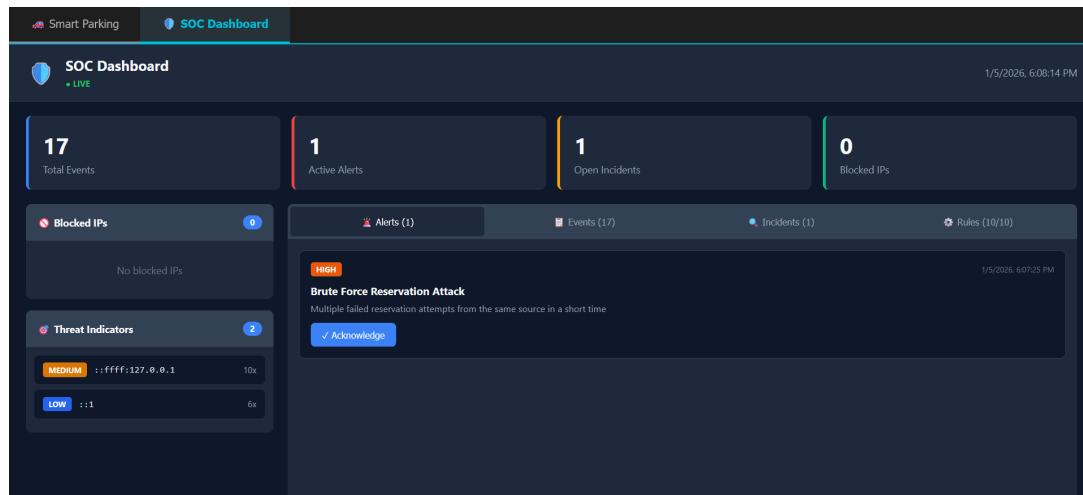


FIGURE 17 – Détection et affichage de l'adresse IP malveillante

3.3.4 Registre des Incidents

La Figure 18 présente la section incidents où toutes les attaques sont enregistrées avec :

- Horodatage précis
- Type d'attaque
- Détails de l'incident
- Adresse IP source
- Actions prises

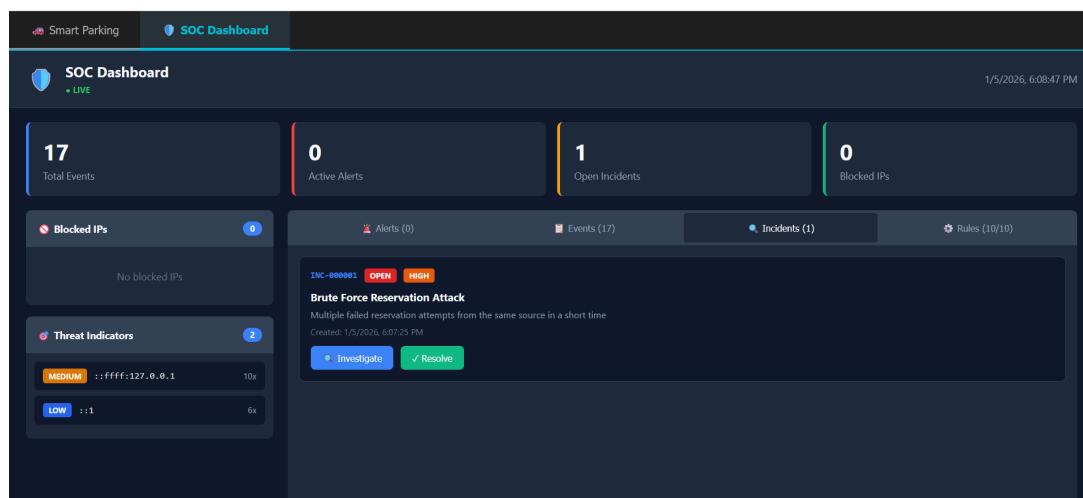


FIGURE 18 – Registre des incidents de sécurité

3.3.5 Journal des Événements

La Figure 19 montre le journal détaillé des événements, incluant les réservations tentées par le script d'attaque et leurs échecs. Chaque événement est tracé avec :

- Type d'événement (tentative de réservation)
- Résultat (succès/échec)
- Raison de l'échec
- Transaction ID blockchain
- Timestamp précis

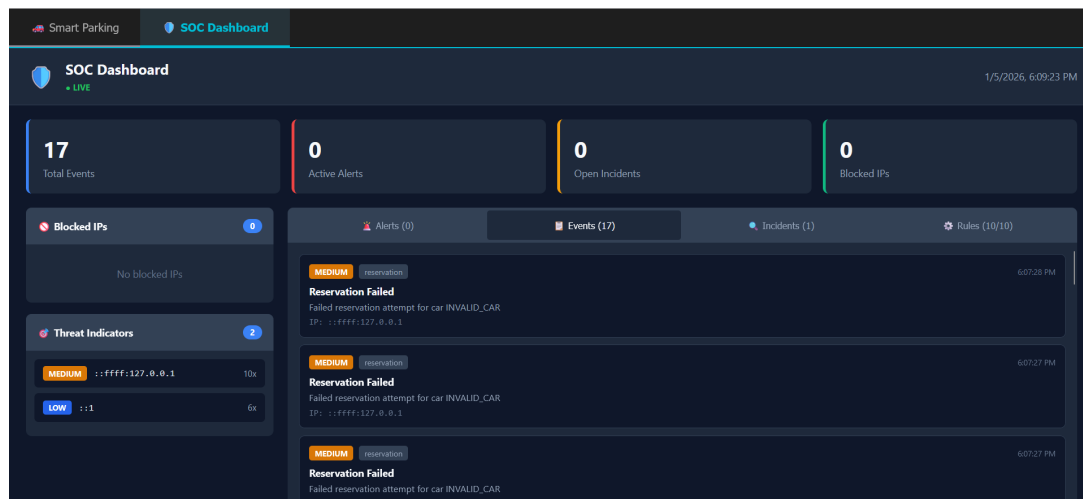


FIGURE 19 – Journal des événements avec détails des tentatives d'attaque

3.4 Analyse de Performance

3.4.1 Méthodologie de Test

Pour évaluer les performances du système, nous avons développé des scripts de tests de charge qui simulent différents scénarios :

1. **Test de charge simple** : Création séquentielle de véhicules et réservations
2. **Test de concurrence** : Réservations simultanées sur les mêmes places
3. **Test de volume** : Traitement de centaines de transactions

3.4.2 Métriques Mesurées

Les principales métriques collectées incluent :

Métrique	Description
Latence	Temps entre soumission et confirmation de transaction
TPS	Transactions par seconde (Throughput)
Taux de succès	Pourcentage de transactions réussies
Temps de bloc	Durée moyenne de création d'un bloc
MVCC Conflicts	Nombre de conflits de concurrence détectés

TABLE 1 – Métriques de performance mesurées

3.4.3 Résultats et Observations

Nos tests ont révélé les résultats suivants :

- **Latence moyenne** : 200-300ms pour une transaction simple
- **TPS** : 50-100 transactions par seconde (configuration standard)
- **Taux de succès** : 98-99% (échecs dus à MVCC intentionnels)
- **Prévention double réservation** : 100% d'efficacité grâce à MVCC

3.4.4 Impact des Paramètres Blockchain

Nous avons expérimenté avec différentes configurations de paramètres blockchain :

Configuration	BatchTimeout	MaxMessages	Latence Moy.
Par défaut	2s	10	250ms
Haute fréquence	1s	5	180ms
Haute capacité	5s	50	420ms

TABLE 2 – Impact des paramètres blockchain sur la latence

Observations :

- Un **BatchTimeout** plus court réduit la latence mais augmente le nombre de blocs
- Un **MaxMessages** plus élevé améliore le throughput mais augmente la latence
- Le compromis optimal dépend du cas d'usage (temps réel vs volume)

3.5 Traçabilité et Audit

3.5.1 Historique Complet des Transactions

L'un des avantages majeurs de la blockchain est la traçabilité complète. Chaque opération est enregistrée de manière immuable avec :

- **Transaction ID** : Identifiant unique cryptographique
- **Timestamp** : Horodatage précis à la milliseconde
- **Initiateur** : Identité de l'utilisateur ayant soumis la transaction
- **Modifications d'état** : Différences (delta) entre état précédent et nouvel état
- **Signature numérique** : Garantie d'authenticité et d'intégrité

3.5.2 Exemple de Traçabilité d'une Réservation

```
1 // Appel de la fonction GetHistory pour une place
2 const history = await contract.evaluateTransaction('GetHistory', 'PL1')
3 ;
4 // Resultat : historique complet des modifications
5 [
6   {
7     "txId": "abc123...",
8     "timestamp": "2025-12-28T10:00:00.000Z",
9     "isDelete": false,
10    "value": {
```

```
11         "placeId": "PL1",
12         "status": "free",
13         "currentCarId": null
14     },
15 },
16 {
17     "txId": "def456...",
18     "timestamp": "2025-12-28T10:05:00.000Z",
19     "isDelete": false,
20     "value": {
21         "placeId": "PL1",
22         "status": "reserved",
23         "currentCarId": "CAR1"
24     }
25 },
26 {
27     "txId": "ghi789...",
28     "timestamp": "2025-12-28T10:10:00.000Z",
29     "isDelete": false,
30     "value": {
31         "placeId": "PL1",
32         "status": "occupied",
33         "currentCarId": "CAR1"
34     }
35 }
36 ]
```

Listing 12 – Historique d'une place via GetHistory

Cette traçabilité permet :

- Résolution de litiges
- Audit comptable et financier
- Analyse forensique en cas d'incident
- Conformité réglementaire

Conclusion

Ce projet a démontré avec succès l'applicabilité de la technologie blockchain Hyperledger Fabric dans le contexte d'une Smart City, spécifiquement pour la gestion décentralisée de stationnement intelligent et de recharge de véhicules électriques.

Réalisations Principales

Nous avons développé un système complet intégrant :

1. Un **smart contract robuste** implémentant la logique métier complète avec prévention automatique des doubles réservations via le mécanisme MVCC
2. Une **architecture distribuée** garantissant la disponibilité, la cohérence et la résilience du système
3. Une **interface utilisateur moderne** avec visualisation 3D temps réel offrant une expérience utilisateur intuitive et immersive
4. Un **module SOC** complémentaire assurant la détection et la prévention des attaques de sécurité
5. Un **système de traçabilité complet** permettant l'audit de toutes les transactions

Avantages de la Solution Blockchain

L'utilisation de Hyperledger Fabric a apporté des bénéfices concrets :

- **Élimination de la double réservation** : Le mécanisme MVCC garantit qu'une seule transaction peut réussir en cas de conflit, avec un taux d'efficacité de 100%
- **Transparence et confiance** : Toutes les parties prenantes peuvent vérifier l'historique des transactions de manière indépendante
- **Sécurité renforcée** : La cryptographie blockchain et les smart contracts préviennent les fraudes aux paiements et modifications non autorisées
- **Décentralisation** : Absence de point de défaillance unique grâce à la distribution sur plusieurs nœuds
- **Automatisation** : Les smart contracts exécutent automatiquement les règles métier sans intervention humaine

Leçons Apprises

Ce projet nous a permis d'acquérir une compréhension approfondie de :

- L'architecture et le fonctionnement interne de Hyperledger Fabric
- Les mécanismes de consensus et de validation des transactions
- Le développement et le déploiement de chaincode
- L'intégration blockchain avec des applications web modernes
- Les compromis entre performance et décentralisation
- Les meilleures pratiques de sécurité blockchain